**LAPORAN PRAKTIKUM**
**Sistem Operasi**

**MODUL 07**
**(Semaphore )**

**Disusun Oleh:**
**Ryan Gabriel Togar Simamora (2311104045)**
**Kelas S1SE-07-02**

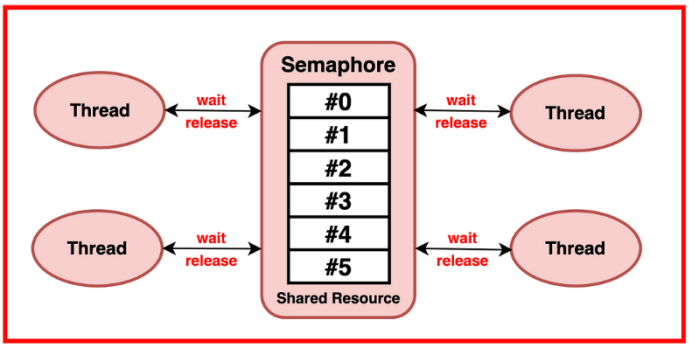
**Dosen : Yudha Islami Sulistya, S.Kom., M.Cs**

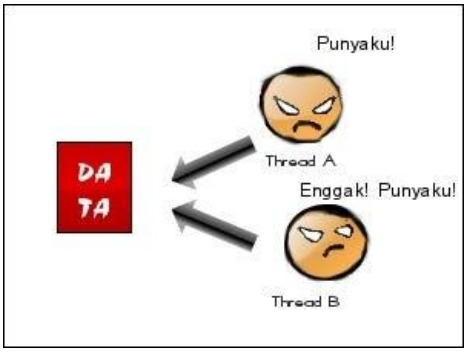
**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK**
**DIREKTORAT KAMPUS PURWOKERTO**
**UNIVERSITAS TELKOM**
**2025**

# BAB I
# PENDAHULUAN

## 1.1 Dasar Teori

### A. Semaphore




**Apa itu semaphrore??**
Sistem operasi Xinu mengandalkan Semaphore sebagai primitif utama untuk mengatur akses ke sumber daya bersama dan mengoordinasikan proses yang berjalan secara multitasking. Artikel ini menjelaskan konsep inti semaphore dalam Xinu dan memandu proses kustomisasi kernel dengan mengintegrasikan implementasi signal()dan mutex() secara langsung ke dalam berkas main.c.

**Konsep Dasar Semaphore dalam Xinu**
Semaphore adalah sebuah primitif sinkronisasi yang beroperasi melalui sebuah variabel integer (counting semaphore) yang mencatat jumlah sumber daya yang tersedia. Nilai integer ini dikelola melalui dua operasi atomik yang krusial:

1. wait() : Operasi yang berusaha mengambil sumber daya. Ini akan mendekrementasi hitungan semaphore. Jika hitungan baru menjadi negatif, proses pemanggil diblokir (status WAIT) dan ditambahkan ke antrean tunggu semaphore.
2. signal() : Operasi yang melepaskan sumber daya. Ini akan menginkrementasi hitungan semaphore. Jika hitungan baru non-positif, ini menandakan ada proses yang menunggu, sehingga satu proses dibangunkan (unblocked) dan dipindahkan ke daftar siap (READY list).

## Tujuan Praktikum:

1. Mahasiswa mampu mengimplementasikan semaphore dengan benar

## Manfaat Praktikum :

1. Memahami

# BAB II
# HASIL PRAKTIKUM

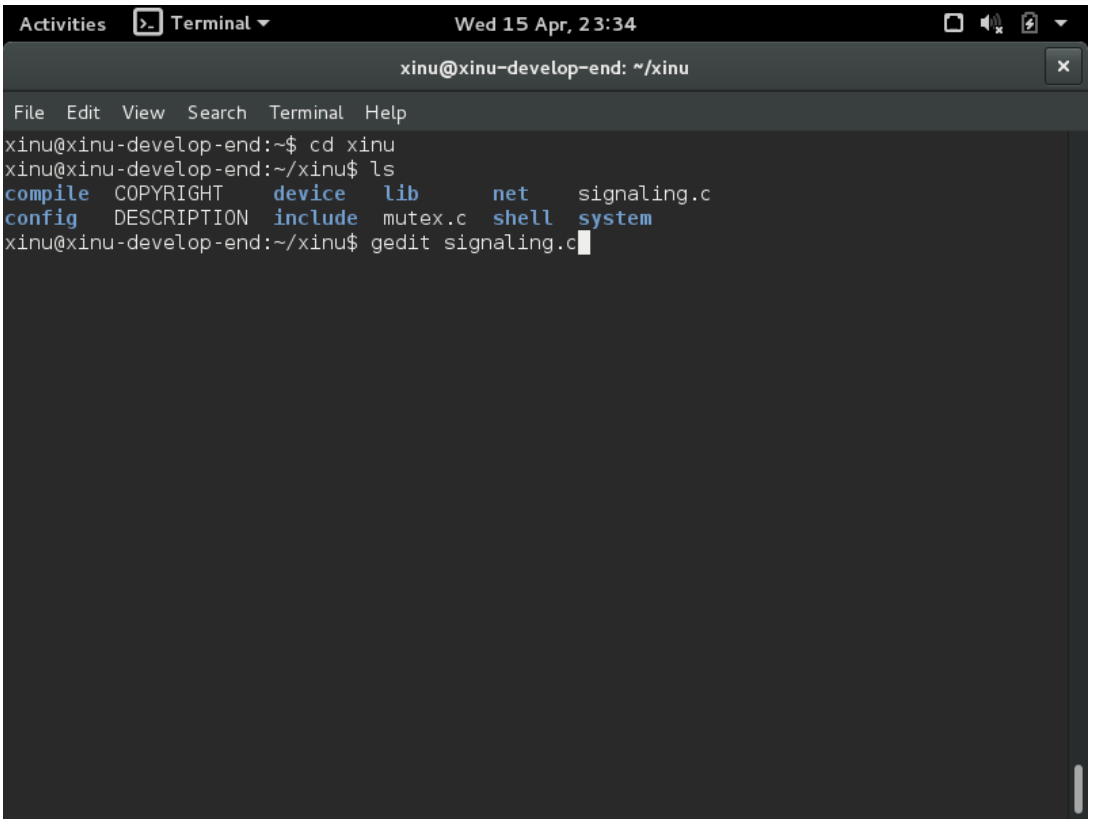
## 2.1 Guided Praktikum

### A. Signaling.c

Pola signaling digunakan sebagai cara untuk memastikan bahwa suatu proses awal (P1) telah selesai dan baru proses selanjutnya (P2) bisa dijalankan. Siganaling mengatur giliran proses.

Implementasi dalam XINU

1. Pertama tama masuk ke direktori xinu dengan perintah cd xinu dan buatkan file baru signaling.c dan langsung edit dengan perintah gedit signaling.c



2. Lalu isi file signaling.c yang baru kita buat seperti pada gambar

```
#include <xinu.h>

void sndA(sid32 s1, sid32 s3);
void sndB(sid32 s2, sid32 s1);
void sndC(sid32 s3, sid32 s2);

process main(void)
{
        sid32 s1, s2, s3;

        s1 = semcreate(1);
        s2 = semcreate(0);
        s3 = semcreate(0);

        resume(create(sndA, 1024, 20, "process A", 2, s1, s3));
        resume(create(sndB, 1024, 20, "process B", 2, s2, s1));
        resume(create(sndC, 1024, 20, "process C", 2, s3, s2));

        return OK;
}

void sndA(sid32 s1, sid32 s3)
{

        while(1)
        {
                wait(s1);
                printf("Selamat datang A\n");
```

3. Lalu Copy kode signaling.c dari direktori xinu, kemudian paste ke main.c di direktori system sehingga kodenya akan seperti di gambar :

```
#include <xinu.h>

void sndA(sid32 s1, sid32 s3);
void sndB(sid32 s2, sid32 s1);
void sndC(sid32 s3, sid32 s2);

process main(void)
{
        sid32 s1, s2, s3;

        s1 = semcreate(1);
        s2 = semcreate(0);
        s3 = semcreate(0);

        resume(create(sndA, 1024, 20, "process A", 2, s1, s3));
        resume(create(sndB, 1024, 20, "process B", 2, s2, s1));
        resume(create(sndC, 1024, 20, "process C", 2, s3, s2));

        return OK;
}

void sndA(sid32 s1, sid32 s3)
{

        while(1)
        {
                wait(s1);
                printf("Selamat datang A\n");
```

4. Lalu klik save dan pergi ke direktori compile, lalu jalankan perintah “make clean” lalu “make” dan masuk ke “sudo minicom” dan jalankan backend-nya.

```
Activities  Terminal  Wed 15 Apr, 23:59
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Hasil Output

```
Activities  Terminal  Thu 16 Apr, 09:50
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help

Xinu for Vbox -- version #112  (xinu)  Thu 16 Apr 02:57:52 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
   4447448 bytes of free memory.  Free list:
           [0x0015E320 to 0x0009FFF7]
           [0x00100000 to 0x005FBFFF]
    115721 bytes of Xinu code.
           [0x00100000 to 0x0011C408]
    148424 bytes of data.
           [0x0011FF00 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)
Selamat datang A
Selamat datang C
Selamat datang B
Selamat datang A
Selamat datang C
Selamat datang B
Selamat datang A
Selamat datang C
Selamat datang B

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:1 | ttyS0
```

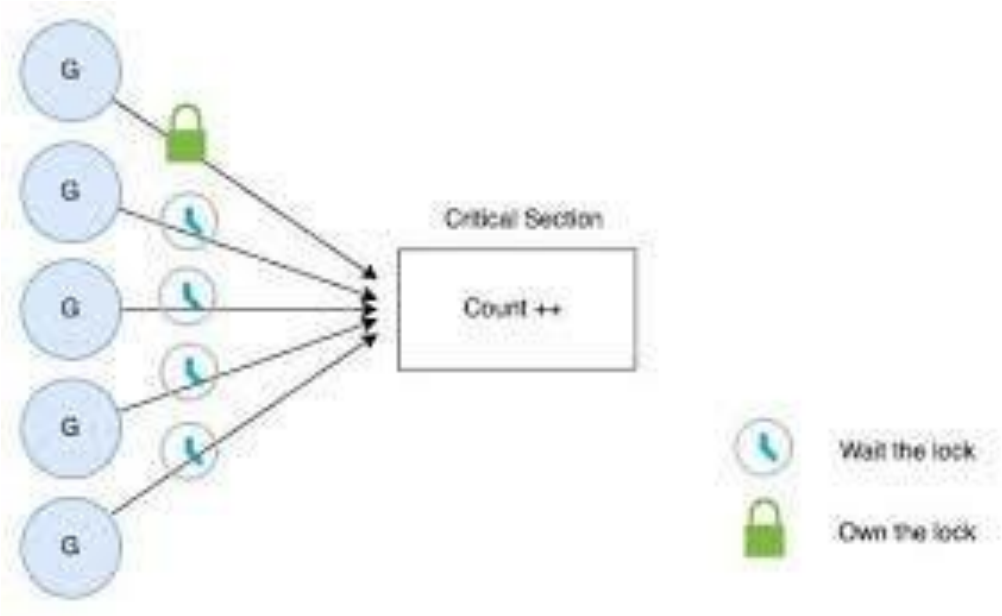
Dengan tiga semaphore s1, s2, s3 kita membuat pola eksekusi:

A -> C -> B -> A -> C -> B ->…
Setiap proses akan berjalan hanya jika semaphore yang ia tunggu bernilai 1. Jika tidak, ia otomatis berhenti.
Contohnya:
A menunggu s1, lalu memberi sinyal ke C
C menunggu s3, lalu memberi sinyal ke B
B menunggu s2, lalu memberi sinyal kembali ke A

Pola ini menciptakan "sirkuit" yang terus berputar.

**B. Mutex.(Mutual Exclusion)**

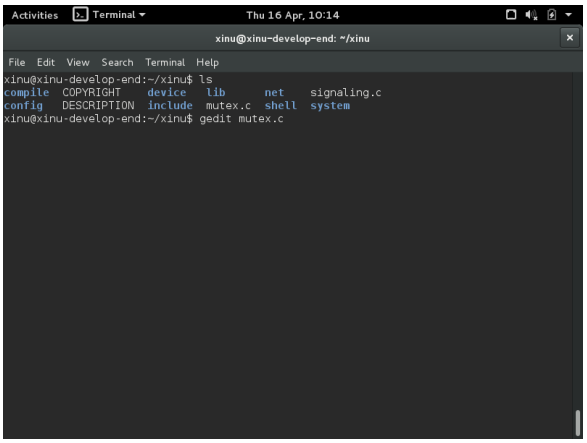
Mutex adalah mekanisme yang memastikan hanya satu proses dapat mengakses suatu resource pada waktu yang sama.



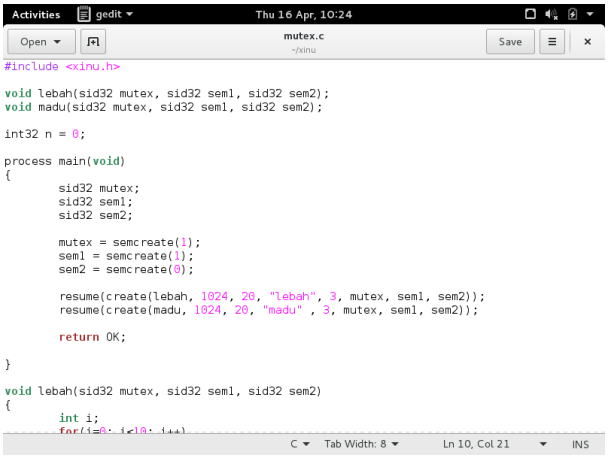

Cara Kerja Mutex

Implementasi dalam XINU

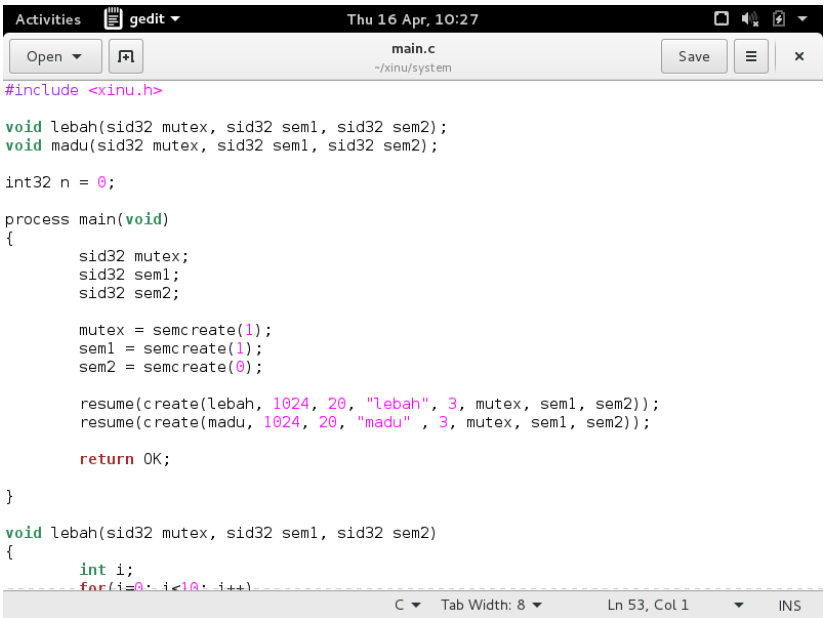
1. Pertama tama masuk ke direktori xinu dengan perintah cd xinu dan buatkan file baru mutex.c dan langsung edit dengan perintah mutex.c




2. Lalu isi file signaling.c yang baru kita buat seperti pada gambar




3. Lalu Copy kode signaling.c dari direktori xinu, kemudian paste ke main.c di direktori system sehingga kodenya akan seperti di gambar :



4. Lalu klik save dan pergi ke direktori compile, lalu jalankan perintah “make clean” lalu “make” dan masuk ke “sudo minicom” dan jalankan backend-nya.

```
Activities  Terminal  Thu 16 Apr, 10:29
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Hasil Output

```
Activities  Terminal  Fri 17 Apr, 09:07
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Help
    115689 bytes of Xinu code.
           [0x00100000 to 0x0011C3E8]
    148424 bytes of data.
           [0x0011FF00 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)
Lebah membuat madu ke 1
Madu tersedia sebanyak 2 buah
Lebah membuat madu ke 3
Madu tersedia sebanyak 4 buah
Lebah membuat madu ke 5
Madu tersedia sebanyak 6 buah
Lebah membuat madu ke 7
Madu tersedia sebanyak 8 buah
Lebah membuat madu ke 9
Madu tersedia sebanyak 10 buah
Lebah membuat madu ke 11
Madu tersedia sebanyak 12 buah
Lebah membuat madu ke 13
Madu tersedia sebanyak 14 buah
Lebah membuat madu ke 15
Madu tersedia sebanyak 16 buah
Lebah membuat madu ke 17
Madu tersedia sebanyak 18 buah
Lebah membuat madu ke 19
Madu tersedia sebanyak 20 buah

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Mengapa Outputnya seperti itu?
Saat dijalankan, Lebah dan Madu tampak seperti bergantian menaikkan nilai n. Padahal, mereka tidak "tahu" bahwa harus bergantian.
OS-lah yang memaksa mereka bergiliran dengan memindahkan proses ke antrean setiap kali mutex dikunci.

Ritmenya menjadi seperti ini:
Lebah masuk → memanggil wait(mutex) → mutex terkunci
Lebah mengubah variabel → keluar
Mutex dibuka oleh Lebah
Madu ditarik dari antrean → masuk ke critical section
Ulangi
Hasil akhirnya adalah eksekusi variabel yang aman, tanpa race condition.

## 2.2 Jurnal Praktikum

1. [50 poin] Buatlah 3 buah proses yaitu P1, P2 dan P3. P1 selalu menampilkan "kwak",
P2 selalu menampilkan "kwik", P3 selalu menampilkan "kwek". Menggunakan 3 proses tersebut dan beberapa buah semaphore, buatlah program yang dapat menampilkan:
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
…

Jawab :

Untuk Mengerjakan soal ini saya pergi ke direktori xinu lalu ke direktori system dan ketikkan perintah gedit main.c lalu kode main.c nya ini saya ambil dari file signaling.c yang pernah dibuat di bagian guided dan sya modifikasi sesuai yang diminta pada soal sehingga kodenya akan seperti di gambar :

```
#include <xinu.h>

void p1(sid32 s1, sid32 s2);
void p2(sid32 s2, sid32 s3);
void p3(sid32 s3, sid32 s1);

process main(void)
{
        sid32 s1, s2, s3;

        s1 = semcreate(1);
        s2 = semcreate(0);
        s3 = semcreate(0);

        resume(create(p1, 1024, 20, "P1", 2, s1, s2));
        resume(create(p2, 1024, 20, "P2", 2, s2, s3));
        resume(create(p3, 1024, 20, "P3", 2, s3, s1));

        return OK;
}

void p1(sid32 s1, sid32 s2)
{

        while(1)
        {
                wait(s1);
                printf("Kwak\n");
```

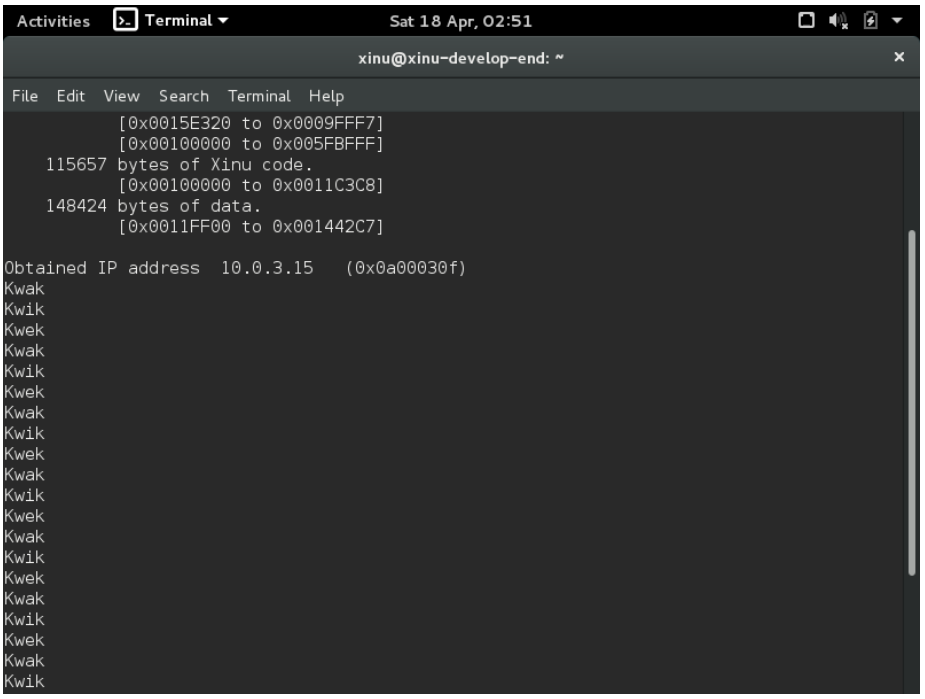
Lalu klik save dan pergi ke direktori compile, lalu jalankan perintah “make clean” lalu “make” dan masuk ke “sudo minicom” dan jalankan backend-nya.

```
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

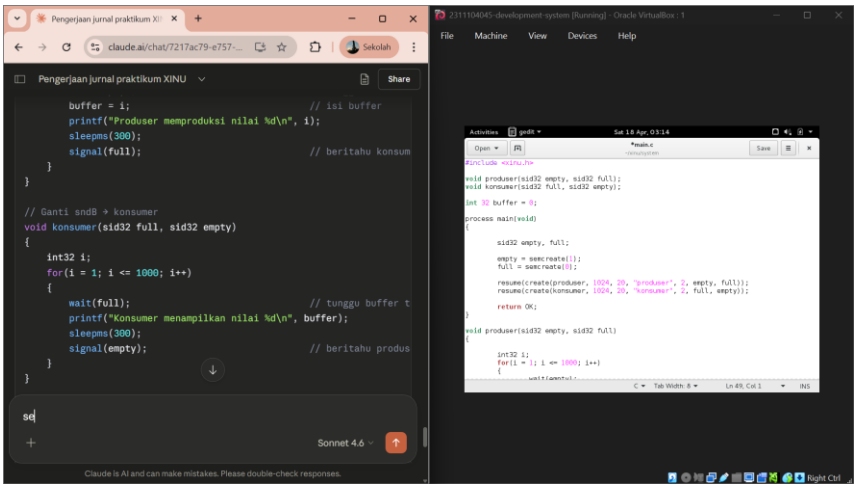
Hasil Output :



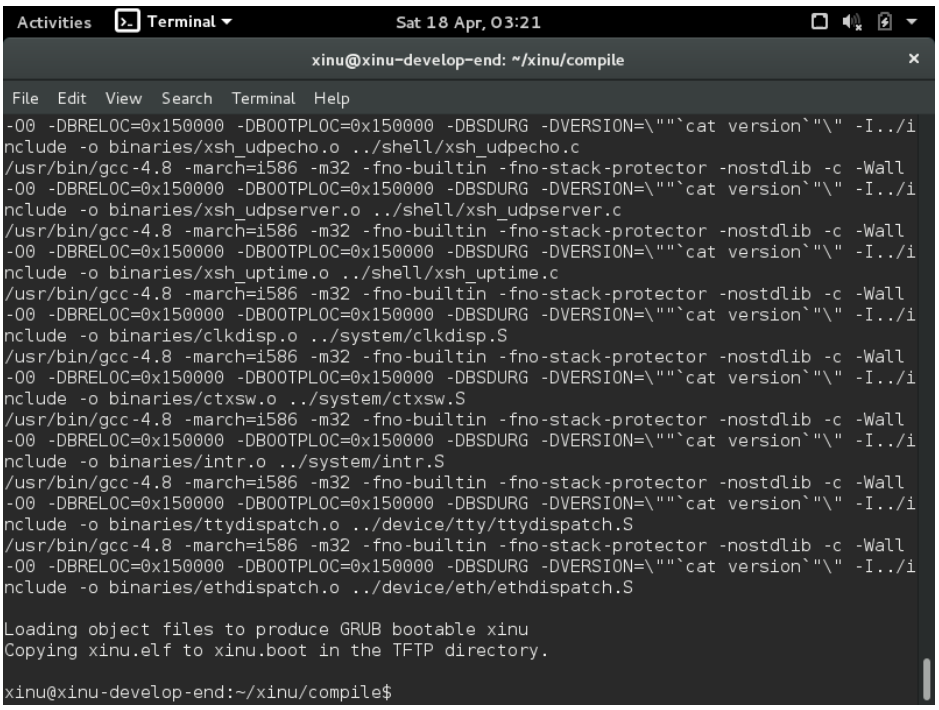

2. [50 poin] Buatlah proses bernama produser yang memproduksi bilangan 1-1000. Buatlah proses bernama konsumer yang akan menampilkan nilai yang diproduksi oleh
produser. Gunakan semaphore!
Produser memproduksi nilai 1
Konsumer menampilkan nilai 1
Produser memproduksi nilai 2
Konsumer menampilkan nilai 2
….
Produser memproduksi nilai 1000
Konsumer menampilkan nilai 1000
Catatan: harus menggunakan semaphore, proses konkuren, tidak boleh hanya 1 proses
dan hanya menggunakan while loop kemudian print.

Jawab :

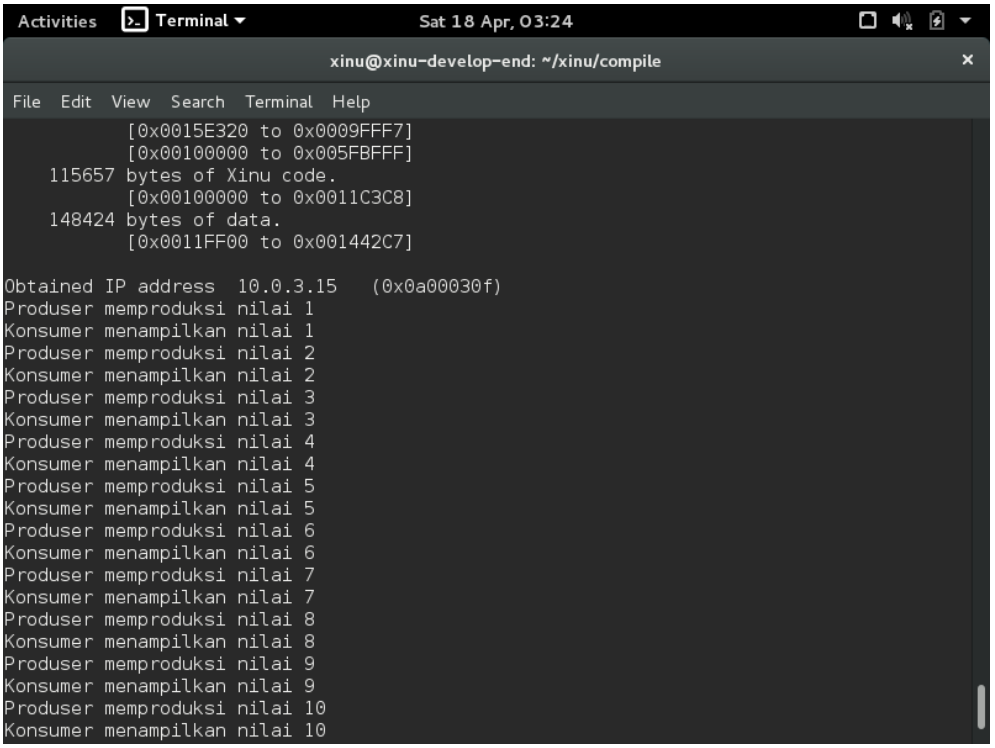
Untuk Mengerjakan soal ini saya pergi ke direktori xinu lalu ke direktori system dan ketikkan perintah gedit main.c  lalu kode main.c nya ini saya ambil dari claude ai dan sya ketik sesuai dengan yang diminta pada soal sehingga kodenya akan seperti di gambar :




Lalu klik save dan pergi ke direktori compile, lalu jalankan perintah “make clean” lalu “make” dan masuk ke “sudo minicom” dan jalankan backend-nya.



Hasil Output :




Ini aku ss sampe nilai 10 kalau nunggu sampai 1000 kelamaan yang penting jalan

# BAB III
# PENUTUP

## Kesimpulan

Melalui praktikum semaphore, mutex, dan signaling di Xinu, kita dapat memahami bahwa sinkronisasi adalah kunci dari permasalahan *race condition*. Di Xinu, kita bisa melihat bagaimana OS benar-benar menghentikan proses, memindahkannya ke antrean, lalu membangunkannya kembali hanya dengan dua operasi sederhana: wait() dan signal().
Mutex mengajarkan bagaimana OS menjaga keamanan data, sementara signaling menunjukkan bagaimana OS mengatur urutan proses. Keduanya dibangun dari mekanisme dasar semaphore, tetapi efeknya sangat berbeda.Semoga dokumentasi ini dapat memberikan gambaran yang jelas dan bermanfaat bagi pembelajaran selanjutnya.